

ВСП тема 2 задание 2.2 часть 2

Справочник по математическим объектам и их представлению в Scilab

Будем называть **объектами** типы данных, которые в почти всегда используются для проведения математических расчётов. Такие объекты существуют в Scilab. Предопределение их внутренней структуры, а также работу памяти среда Scilab выполняет самостоятельно, без участия пользователя. Однако эти предопределённые объекты могут быть использованы для создания своих, пользовательских объектов.

В основном объектами являются массивы с данными. В этом разделе будет показано, как создавать и манипулировать основными предопределёнными объектами.

Переменная в Scilab (от англ. variable) — тот же массив, только всего с одним полем для данных разного типа.

С созданием переменной справится каждый, для этого необходимо и достаточно придумать для нее имя и присвоить ей любое начальное значение. Выделение памяти и область видимости(локальная) среда берёт на себя.

Однако существуют некоторые правила, которым должно удовлетворять наименование переменной:

1. Имя переменной может состоять из букв латинского алфавита обоих регистра и/или из цифр. Стоит отметить, что регистр имеет значение! Например AB,ab,Ab,aB – четыре совершенно разных переменных.
2. Имя переменной не может начинаться с цифры, однако может состоять и начинаться с '!', '#', '\$', '%', '?', '_'
3. Совпадение имен переменной с зарезервированными словами, функциями и т.п. недопустимо.

Создадим четыре переменные:

```
num1=5,num2=-39,2334,num3=%pi*3.34e-5,num4=3-%i*5
```

%pi означает число пи, %i – мнимая единица. Как нетрудно догадаться, % означает обращение к предопределённой в среде константе.

Целый тип данных

В среде Scilab целые числа можно создавать только при помощи специальных функций. В остальных случаях числовое значение будет всегда иметь вещественный тип данных.

В зависимости от задачи, возможно использование разных типов целочисленных переменных. Существует разделение на знаковые и беззнаковые переменные- signed и

unsigned (с англ. беззнаковый). В unsigned количество бит, отведенные под знак в signed типе используется для расширения диапазона допустимых значений. Другими словами, unsigned – неотрицательные числа. Другим параметром типа является количество бит, отведенных для хранения целочисленной переменной: их может быть 8, 16, 32. От количества бит зависит диапазон допустимых чисел. На следующей фотографии приведены функции для создания целых чисел и диапазоны допустимых значений, все функции возвращают целое число, указанное в качестве аргумента функции:

Имя функции	Тип целого числа	Диапазон	Код представления
int8()	знаковое 8-битное число	$[-2^7, 2^7 - 1] = [-128, 127]$	1
uint8()	беззнаковое 8-битное число	$[0, 2^8 - 1] = [0, 255]$	11
int16()	знаковое 16-битное число	$[-2^{15}, 2^{15} - 1] = [-32768, 32767]$	2
uint16()	беззнаковое 16-битное число	$[0, 2^{16} - 1] = [0, 65535]$	12
int32()	знаковое 32-битное число	$[-2^{31}, 2^{31} - 1] = [-2147483648, 2147483647]$	4
uint32()	беззнаковое 32-битное число	$[0, 2^{32} - 1] = [0, 4294967295]$	14

Для работы с целочисленными переменными можно использовать две функции:

1. `iconvert(X, itype)` — меняет представление в памяти в общем случае матрицы из вещественных или целых чисел. Аргументы функции: X- имя этой матрицы, itype- тип целого числа
2. `inttype()` — возвращает код внутреннего представления в общем случае матрицы целочисленных данных.

Матрицы и векторы

Вектор в Scilab — одномерный массив одного типа данных. В линейной алгебре существуют вектор-строки и вектор-столбцы, от этого зависит применимость некоторых математических операций. Например, скалярное произведение возможно тогда и только тогда, когда первый множитель является строкой, а второй — столбцом. Вернёмся к Scilab. Как возможно некоторым известно, у каждого элемента массива есть свой порядков номер или, как его называют, индекс. В отличие от привычного нуля из программирования, индексирование элементов в Scilab начинается с единицы.

Чтобы создать вектор, необходимо дать ему имя в соответствии с правилами, описанными выше и присвоить вектору значение, заключив все в квадратные скобки, перечисляя элементы через разделитель (запятую или пробел):

```
--> VECTOR = [1, 2, 3, 4, 5]
```

Чтобы создать вектор-столбец, достаточно отделить элементы точкой с запятой:

```
-->[1; 2; 3]
ans =
  1.
  2.
  3.
```

Матрицы

Матрица в Scilab — двумерный массив, состоящий из одного типа данных. Матрицу можно представить в качестве нескольких векторов строк, записанных столбцов.

Способы создания матриц:

- Матрицу можно создать из составляющих ее элементов;
- Из имеющихся векторов, упорядочив их строками или столбцами;
- Одной из специальных функций.

В общем случае ее можно представить в виде:

$[a_{11}, a_{12}, a_{13}; a_{21}, a_{22}, a_{23}; a_{31}, a_{32}, a_{33}]$ – матрица размером 3 на 3

Запятая, как и в случае с вектором, не обязательна:

```
-->A=[1 2; 3 4]
A =
    1.    2.
    3.    4.
```

Функции

Функции разделяют на два типа: программируемая функция – некоторый алгоритм действий, математическая функция – отражение связи между множеством аргументов и множеством значений функции. Функцию в Scilab можно создать самому(программируемые) или воспользоваться уже готовыми.

Математическую функцию можно описать при помощи специальной функции `deff()`. Общий вид этой функции:

```
deff(' [rv1,rv2,rv3,...]=funName(x1,x2,...) ', ['rv1=выражение1'; 'rv2=выражение 2'; и т.д])|
```

где `rv`- возвращаемое значение, `y` в обычном понимании, `funName`- имя функции(сами можем назвать).

Пример использования `deff()`

```
--> deff('y=f(x) ', 'y=x+5')
```

```
--> x=5, y=f(x)
```

```
x =
```

```
5.
```

```
y =
```

```
10.
```

Также существует функция `funcprot()`-защита функции от переопределения.

В зависимости от аргумента, переданного функции, она будет работать по разному:

- 0 — отключение механизма защиты;
- 1 — формирование предупреждения (используется по умолчанию);
- 2 — формирует ошибку при попытке переопределения существующей функции.

Казалось бы, на этом с функциями можно закончить. Но есть одно но: `deff()` может принимать бесконечного много инструкций, но при большом их количестве понять что делает функция становится очень сложно. Чтобы исправить эту проблему, создали вторую конструкцию: `function ... endfunction`. Общий вид:

```
function <выходные переменные>=имя_функции(аргументы)
...
инструкции
...
endfunction
```

Данная конструкция чаще всего используется для написания программируемых функций из-за простоты. Хотя никто не запрещает использовать `deff()`. Замечательное свойство интерпретатора Scilab заключается в том, что можно ввести первую строку с ключевым словом `function`, именем функции, аргументами и входными переменными, после этого нажать ENTER и далее интерпретатор будет воспринимать все, что вводится далее, как инструкции для функции. Чтобы закончить вводить инструкции, нужно ввести ключевое слово `endfunction`. Если в теле функции не допущено синтаксических ошибок, то функцию можно использовать, иначе вы увидите ошибку, и тогда придется все начинать с самого начала.